K.R. MANGALAM UNIVERSITY

Gurugram, Haryana

Department of Computer Science & Engineering

ASSIGNMENT 1

Deconstructing System Performance – From Moore to IPC

1. Model Creation: RISC Abstraction Stack

A modern RISC processor based on Von Neumann architecture is best understood through a layered abstraction model. Each layer hides implementation complexity from the layer above it, enabling modular design, portability, and reuse. The figure below presents the complete 10-layer abstraction stack.

Figure 1: Complete RISC Abstraction Stack (Von Neumann Architecture)

1.1 Layer-by-Layer Analysis

The abstraction stack progresses from physical electrons at the lowest level to user-facing applications at the highest. The critical boundary is the Instruction Set Architecture (ISA), which acts as the contract between hardware and software.

- Layers 10–8 (Application → HLL): Software-level abstractions written by programmers
- Layer 7 (OS/Runtime): Manages resources and provides system calls; isolates apps from hardware
- Layer 6 (RISC ISA): Fixed-width 32-bit instructions (LOAD, STORE, ADD, SUB, BRANCH); few addressing modes
- Layer 5 (Microarchitecture): Implements the ISA via a 5-stage pipeline: IF→ID→EX→MEM→WB
- Layers 4–2 (RTL → Transistors): Physical implementation in CMOS silicon
- Layer 1 (Physics): Quantum tunnelling and semiconductor band-gap physics

1.2 Von Neumann vs Harvard Comparison

RISC processors traditionally follow the Von Neumann model (shared instruction/data memory bus) but modern implementations use a Harvard cache structure internally (separate L1-I and L1-D caches) to overcome the Von Neumann bottleneck while maintaining a unified physical memory model.

2. IPC Benchmark Study

2.1 Processor Specifications

Parameter	Proc-A (In-Order RISC)	Proc-B (OoO Superscalar)
Architecture	In-Order, 4-stage	Out-of-Order, 8-wide
Clock Speed	1.2 GHz	3.6 GHz
Issue Width	1 instruction/cycle	Up to 4 instructions/cycle
Branch Prediction	Static (not taken)	Dynamic (tournament predictor)
Reorder Buffer	None	256-entry ROB

2.2 IPC Calculation Methodology

IPC (Instructions Per Cycle) is the primary architectural throughput metric. Three related metrics are defined:

Metric	Formula	Interpretation
IPC	IPC = Instructions ÷ Cycles	Higher = better throughput
CPI	CPI = Cycles ÷ Instructions	Lower = fewer stalls
Execution Time	T = Cycles ÷ Clock Frequency	Lower = faster execution

2.3 Benchmark Data and Results

Workload	Processor	Instructions	Cycles	IPC	Exec Time (ms)
Compute-Heavy	Proc-A	2,000,000	2,400,000	0.833	2.000
Compute-Heavy	Proc-B	2,000,000	1,200,000	1.667	0.333
I/O-Heavy	Proc-A	800,000	3,200,000	0.250	2.667
I/O-Heavy	Proc-B	800,000	2,400,000	0.333	0.667

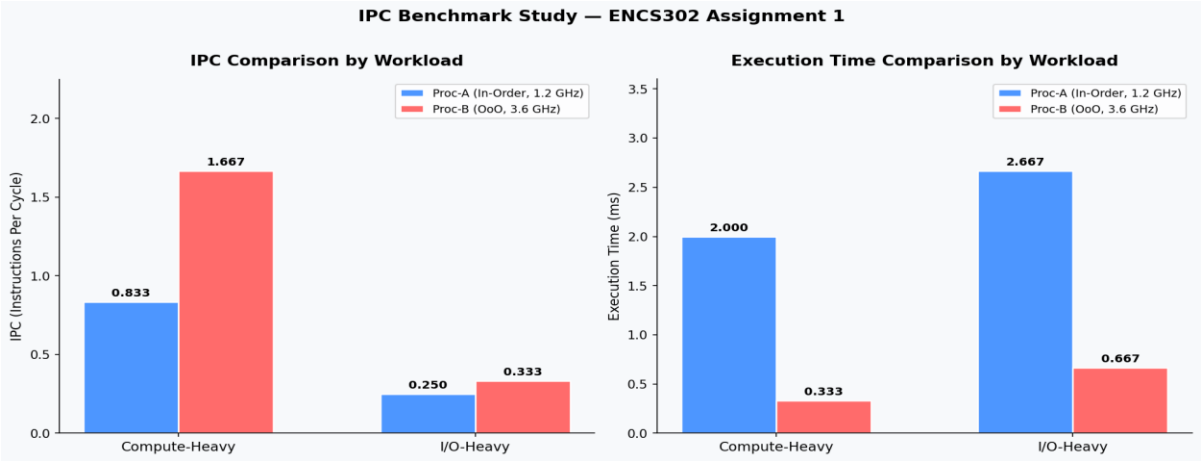

Figure 2: IPC and Execution Time Comparison — Compute-Heavy vs I/O-Heavy Workloads

2.4 Analysis and Conclusions

Compute-Heavy Workload

Proc-B achieves an IPC of 1.667 vs 0.833 for Proc-A — a 2× throughput advantage. The out-of-order engine finds independent ALU operations and issues them simultaneously. Combined with its 3× higher clock (3.6 GHz vs 1.2 GHz), Proc-B executes the compute benchmark 6× faster (0.333 ms vs 2.000 ms). This demonstrates the multiplicative benefit of IPC × Clock in compute-bound scenarios.

I/O-Heavy Workload

Under I/O-heavy conditions, IPC drops sharply for both processors: Proc-A falls to 0.250 (CPI = 4.0) and Proc-B to 0.333 (CPI = 3.0), revealing severe memory stall penalties. The I/O bottleneck dominates because the pipeline stalls waiting for slow storage/network devices. Proc-B still wins in absolute execution time (0.667 ms vs 2.667 ms) due to its higher clock frequency, but the architectural IPC advantage narrows significantly.

- Key Insight 1: IPC is workload-dependent. Superscalar OoO designs shine in compute-bound code.
- Key Insight 2: Memory/I/O stalls are the dominant performance limiter in I/O-heavy workloads.
- Key Insight 3: Execution Time = (Instructions × CPI) / Clock; improving any factor reduces time.

3. Moore's Law Plot (2009–2024)

Gordon Moore's 1965 observation — that transistor counts double approximately every two years — has been the dominant driver of computing performance for six decades. The log-scale plot below charts real chip data from 2009 to 2024.

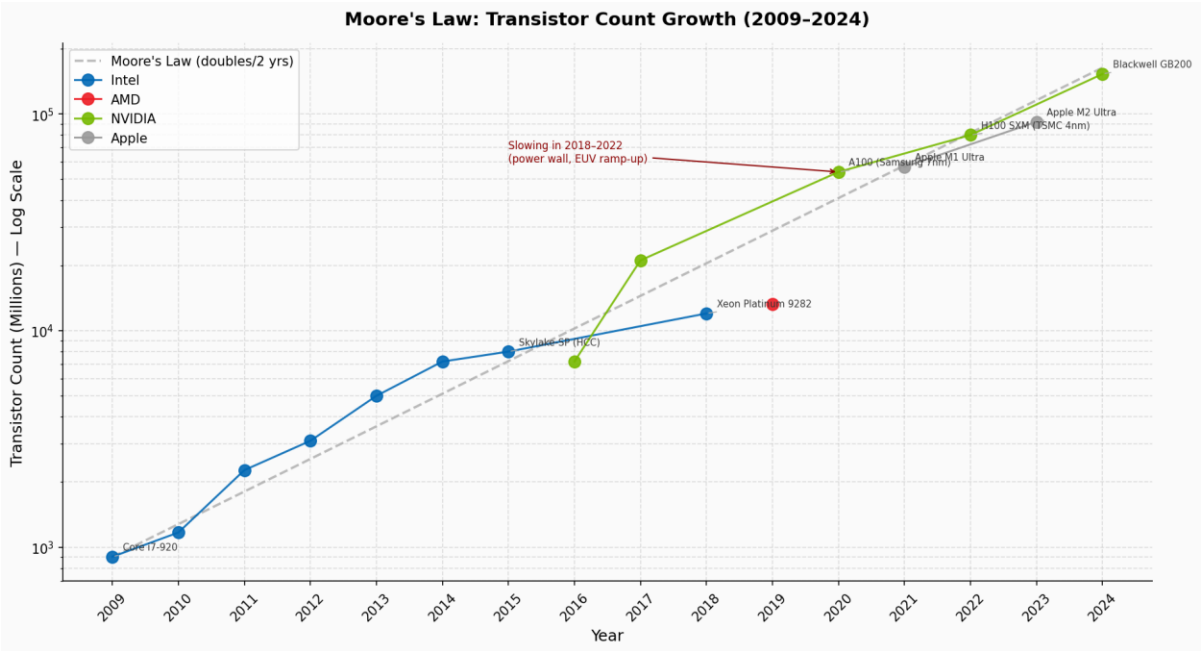

Figure 3: Moore's Law — Real Transistor Count Data (2009–2024) on Log Scale

3.1 Key Observations

- 2009–2017: Moore's Law held well. Intel's shrinks from 45nm to 14nm and NVIDIA's GPU scaling kept pace with the theoretical doubling curve.
- 2018–2022: The pace slowed visibly. Power walls, EUV lithography ramp-up challenges, and rising manufacturing costs caused Intel to stall on 14nm for multiple generations.
- 2020–2024: TSMC's 7nm/5nm/4nm nodes enabled explosive GPU transistor counts (A100: 54B, H100: 80B, GB200: 153B). GPUs now vastly outpace CPUs in raw transistor count.
- 2024 Status: NVIDIA's Blackwell GB200 at 153 billion transistors on TSMC 4nm exceeds Moore's classical projection by ~3×, enabled by chiplet architectures and advanced packaging (CoWoS).

3.2 Post-Moore Era Strategies

As dimensional scaling slows below 2nm gate lengths, the industry has pivoted to architectural and packaging innovations:

- 3D stacking (HBM memory, V-cache): vertical integration multiplies effective density
- Chiplets / MCM: AMD Zen CPUs combine multiple dies in one package
- Specialised silicon: TPUs, NPUs, DPUs trade generality for task-specific efficiency
- RISC-V open ISA: reduces NRE costs for custom chip design

4. Trade-off Analysis: Cost vs Performance vs Abstraction

4.1 The Performance-Cost Triangle

Every processor design lives inside a three-way tension between raw performance, manufacturing cost, and architectural abstraction depth. No design can simultaneously optimise all three. Understanding these trade-offs is fundamental to computer architecture decision-making.

4.2 Performance vs Cost

High-performance processors (e.g., Intel Core i9, AMD EPYC, NVIDIA H100) achieve their speed through deep out-of-order pipelines, wide superscalar issue, multi-level cache hierarchies, and advanced branch predictors. Each of these features requires additional transistors, which increases die area. Larger dies are more expensive per wafer, have lower yield (more defects per chip), and consume more power — requiring expensive cooling solutions.

Conversely, embedded RISC-V or ARM Cortex-M0 processors use in-order 2-stage pipelines with no branch prediction, yielding tiny die areas under 0.1 mm^2 that cost pennies to manufacture and run on microwatts of power. The performance gap exceeds 1,000× compared to a server CPU, but the cost gap is equally dramatic: a high-end Xeon costs $5,000+ while a Cortex-M0 MCU costs $0.50.

4.3 Abstraction vs Performance

Abstraction layers — OS, runtime, virtual machine, interpreter — provide portability, security isolation, and developer productivity. However, each abstraction layer introduces overhead. A Java program running in a JVM executes several times fewer raw instructions per second than equivalent code compiled to native machine code, because the JVM's just-in-time compiler, garbage collector, and object model all consume cycles.

RISC architectures deliberately minimise hardware abstraction at the ISA level, exposing simple operations (load, store, add, branch) that compile efficiently. This philosophy underpins the performance of Apple's M-series and ARM Cortex-A chips which dominate mobile and laptop performance-per-watt rankings, outperforming more complex CISC designs in many workloads.

4.4 Quantified Trade-off Summary

Design Point	Performance	Cost	Abstraction Overhead
HPC GPU (H100)	Extreme (80 TFLOPS)	Very High (~$30,000)	Low (bare-metal CUDA)
Server CPU (EPYC)	High (multi-core OoO)	High (~$5,000)	Medium (OS + hypervisor)
Mobile SoC (Apple M4)	High perf/watt	Medium (~$200)	Medium (macOS/iOS)
RISC MCU (Cortex-M0)	Low (single-core)	Very Low (~$0.50)	Minimal (bare-metal)

4.5 Design Implications

- Cloud/HPC: Maximise throughput per dollar; accept high upfront cost. Use OoO superscalar CPUs + GPU accelerators.
- IoT/Embedded: Minimise power and cost; sacrifice performance. Use in-order RISC MCUs with RTOS or bare-metal firmware.
- Consumer Devices: Balance all three. Apple's unified memory architecture (UMA) achieves this by eliminating the CPU-GPU memory copy bottleneck — a key abstraction innovation.
- Data Centres: Workload-specific silicon (TPUs, DPUs, SmartNICs) offloads repetitive tasks from general-purpose CPUs, improving cost-efficiency by 5–10× for targeted workloads.

The convergence of Moore's Law slowdown and rising abstraction complexity means future performance gains will come from domain-specific architectures (DSAs), heterogeneous integration, and near-memory computing rather than traditional transistor scaling alone.

5. Functional Unit Explanation

A Von Neumann computer is composed of four fundamental functional units that interact via a common system bus: the Arithmetic Logic Unit (ALU), Control Unit (CU), Memory subsystem, and Input/Output (I/O) interface. The figure below illustrates all four units and their internal structures.

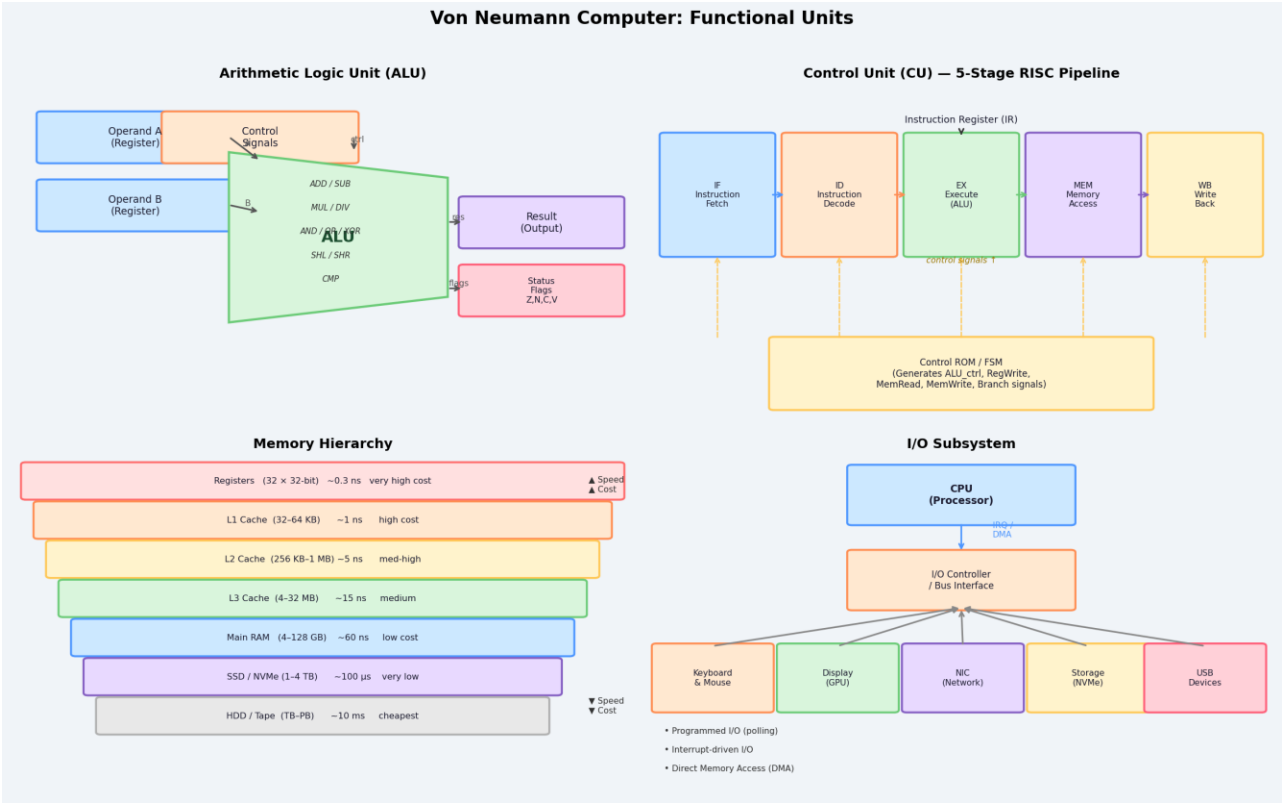

Figure 4: Von Neumann Computer — Four Functional Units with Internal Structures

5.1 Arithmetic Logic Unit (ALU)

The ALU is the computational engine of the processor. It receives two operands from the register file (via the data path) and a set of control signals from the Control Unit specifying the operation to perform. Outputs are a result value written back to a register and a set of status flags (Zero, Negative, Carry, Overflow) used by conditional branch instructions.

- Integer operations: ADD, SUB, MUL, DIV (hardware multiply/divide unit)
- Bitwise logic: AND, OR, XOR, NOT — essential for bit masking and flags
- Shift operations: SHL (shift left = multiply by 2), SHR (shift right = divide by 2)
- Comparison: CMP sets flags without storing result; used by BEQ, BNE, BLT branch instructions

In modern superscalar processors, multiple ALUs operate in parallel (e.g., ARM Cortex-A78 has 4 integer ALUs + 2 FP/SIMD units), enabling instruction-level parallelism.

5.2 Control Unit (CU)

The Control Unit orchestrates all activities in the processor. It fetches instructions from memory (via the Program Counter), decodes their operation code (opcode) and operand fields, and generates control signals that direct every other unit. There are two main CU implementations:

- Hardwired CU: Boolean logic gates directly decode opcodes → fast, but inflexible. Used in RISC processors.
- Microprogrammed CU: A micro-ROM stores control word sequences for each instruction → slower, but easily extended. Common in older CISC designs.

In a 5-stage RISC pipeline (IF → ID → EX → MEM → WB), the CU generates separate control signals for each stage simultaneously, enabling different instructions to occupy different stages concurrently — the foundation of pipelining.

5.3 Memory Subsystem

Memory is organised as a hierarchy trading capacity against access speed. The CPU accesses registers (< 1 ns) for active data, then L1 cache (~1 ns, 95%+ hit rate), L2 cache (~5 ns), L3 cache (~15 ns), and finally main DRAM (~60 ns) on a cache miss. Storage (SSD/HDD) is accessed via the I/O subsystem when the OS handles page faults.

The locality principle underpins cache effectiveness: temporal locality (recently accessed data will be accessed again) and spatial locality (nearby memory is likely to be accessed next). Hardware prefetchers exploit these patterns to bring data into cache before it is needed, reducing effective memory latency.

5.4 Input/Output (I/O) Subsystem

The I/O subsystem connects the processor to the outside world through a standardised bus interface (PCIe, USB, SATA, Ethernet). Three fundamental I/O mechanisms exist:

- Programmed I/O (Polling): CPU continuously checks device status register in a tight loop. Simple but wastes CPU cycles.
- Interrupt-Driven I/O: Device sends an interrupt request (IRQ) to the CPU when ready. CPU handles the interrupt service routine (ISR) then returns to the main program. More efficient.
- Direct Memory Access (DMA): A dedicated DMA controller transfers data directly between device and memory, bypassing the CPU entirely. Used for high-bandwidth transfers (NVMe, GPU, NIC). CPU involvement reduced to setup and completion notification only.

Modern SoCs integrate the I/O controller on-die, reducing latency and improving energy efficiency compared to separate chipsets.

6. Binary and Hexadecimal Conversions

Number system conversions are foundational to computer architecture — memory addresses, instruction encoding, and register values are all represented in binary or hexadecimal. The table below demonstrates seven key conversions including two's complement representation for negative numbers.

Conversion Type	Input Value	From Base	To Base	Result
Decimal → Binary	187	Base 10	Base 2	10111011
Decimal → Hexadecimal	3749	Base 10	Base 16	EA5
Binary → Decimal	11010110	Base 2	Base 10	214
Hexadecimal → Binary	A9F	Base 16	Base 2	101010011111
Hexadecimal → Decimal	3C8	Base 16	Base 10	968
Binary → Hexadecimal	1111000011	Base 2	Base 16	3C3
Two's Complement of −42	−42	Base 10	Base 2	11010110 (8-bit)

6.1 Two's Complement Derivation for −42

- Step 1: Write +42 in 8-bit binary: 00101010
- Step 2: Invert all bits (one's complement): 11010101
- Step 3: Add 1 (two's complement): 11010110
- Verification: 11010110 in unsigned = 214 = 256 − 42 ✓

Two's complement is the universal standard for signed integer representation because it allows addition and subtraction to use the same hardware circuit, and has only one representation of zero.

References

- Patterson, D. A., & Hennessy, J. L. (2020). Computer Organization and Design RISC-V Edition (2nd ed.). Morgan Kaufmann.
- Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
- Moore, G. E. (1965). Cramming more components onto integrated circuits. Electronics, 38(8).
- NVIDIA Corporation. (2024). NVIDIA Blackwell Architecture Technical Brief. NVIDIA Developer.
- AMD. (2023). AMD EPYC 9004 Series Processor Architecture. AMD Technical Brief.
- Apple Inc. (2023). Apple M2 Ultra Chip Architecture. Apple Developer Documentation.
- Intel Corporation. (2023). Intel 12th Gen Core Processor Architecture Overview. Intel Developer Zone.
- Wikichip.org — Chip transistor count database (2009–2024). https://en.wikichip.org